

Aula 1 — Introdução à Análise e Projeto de Sistemas

Sobre a disciplina

Análise e Projeto de Sistemas estuda como transformar uma necessidade real (de uma empresa, de um cliente, de um usuário) em um software bem planejado, documentado e construído de forma organizada — antes de sair escrevendo código.

Ao longo do semestre vamos trabalhar com:

- Levantamento de requisitos e casos de uso
- Modelagem com UML (diagrama de casos de uso, classes e sequência)
- Metodologias ágeis (Scrum e Kanban)
- Versionamento e colaboração em equipe com Git (Conventional Commits, branches, pull requests, code review)

Metodologia: aulas teóricas conectadas a prática. A partir das próximas aulas vamos construir, em conjunto, um projeto de software real — desde a primeira conversa com o "cliente" até o diagrama de sequência — aplicando cada conceito no próprio projeto.

Ferramenta de modelagem: usaremos o [Draw.io](https://draw.io) (gratuito, roda no navegador) para todos os diagramas.

Avaliação:

- Avaliação Regimental (AV1) — 5,0 pontos — cobre todo o conteúdo do semestre
- Avaliações (A2) — 5,0 pontos, distribuídos entre avaliações parciais (2 Provas de 1 ponto e um trabalho de 3 pontos)

Cronograma resumido:

Bloco	Conteúdo
1	Etapas do desenvolvimento de software
2–4	Requisitos, casos de uso e diagrama de casos de uso
5–10	Diagramas de classes

Bloco	Conteúdo
12–13	Modelagem de interações (diagrama de sequência)
14	Metodologias ágeis: Scrum e Kanban
15–16	Git: commits, branches, pull requests e code review

Bibliografia base: WAZLAWICK, *Análise e design orientados a objetos*; PRESSMAN, *Engenharia de Software*; LEDUR, *Análise e projeto de sistemas*.

O que é Análise e Projeto de Sistemas?

Todo software nasce de um problema. Antes de programar, alguém precisa responder duas perguntas separadas:

- **Análise** → *O quê* o sistema precisa fazer? Quem vai usá-lo? Quais regras ele precisa respeitar?
- **Projeto (design)** → *Como* isso vai ser construído? Que classes, telas, banco de dados e arquitetura vão sustentar essas funcionalidades?

Exemplo prático: imagine um app de agendamento para uma clínica.

- Análise: "o paciente precisa marcar consulta escolhendo especialidade e horário disponível"
- Projeto: decidir se vai existir uma classe `Consulta`, relacionada a `Paciente` e `Medico`, com uma tela de calendário que consulta um banco de dados de horários

Pular direto para o código sem passar por essas duas etapas é a principal causa de sistemas que "funcionam", mas são difíceis de manter, cheios de retrabalho e que não atendem de fato o que o cliente pediu.

As etapas do desenvolvimento de software

Todo processo de desenvolvimento — seja cascata, ágil ou qualquer outro — passa, em algum momento, por quatro atividades fundamentais:

1. Especificação

Define **o que** o sistema deve fazer e sob quais restrições (prazo, tecnologia, orçamento, segurança). É aqui que entram os requisitos.

2. Desenvolvimento

O sistema é **projetado e implementado**: arquitetura definida, classes modeladas, telas construídas, código escrito.

3. Validação

Verifica-se se o que foi construído realmente atende ao que foi especificado — testes, revisão com o cliente, correção de falhas.

4. Evolução

O software está em produção, sendo usado — e o mundo muda. Surgem novos requisitos, correções, adaptações. O sistema evolui.

Exemplo prático — um app de delivery:

Etapa	O que acontece
Especificação	Definiu-se que o app precisa conectar restaurantes e clientes, com pagamento online e rastreamento de pedido
Desenvolvimento	Times de back-end e front-end constroem a API, o app do cliente e o painel do restaurante
Validação	Testa-se se um pedido realmente chega ao restaurante certo, se o pagamento é processado corretamente
Evolução	Meses depois, adiciona-se rastreamento do entregador em tempo real, avaliação por estrelas, cupons de desconto

Perceba que a evolução nunca para — é um ciclo que se repete continuamente enquanto o software estiver em uso.

Prática

Escolha um aplicativo que vocês usam no dia a dia (WhatsApp, iFood, Instagram, um sistema da própria faculdade etc.) e identifiquem:

1. Um exemplo de requisito que provavelmente fez parte da **especificação** original
2. Uma decisão de **desenvolvimento** (arquitetura, tecnologia, estrutura) que vocês conseguem observar no uso do app
3. Um indício de **validação** — algo que só funciona porque foi testado (ex: mensagens de erro, confirmações)
4. Uma funcionalidade que foi claramente adicionada depois — um exemplo de **evolução**

Cada dupla apresenta um exemplo rápido para a turma.

Aula 2 — Engenharia de Requisitos e Introdução aos Casos de Uso

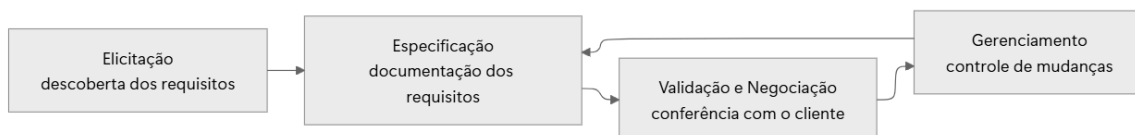
Da especificação aos requisitos

Na aula anterior vimos que todo desenvolvimento de software passa por quatro etapas: especificação, desenvolvimento, validação e evolução. Hoje vamos mergulhar na **especificação** — a etapa em que descobrimos e documentamos o que o sistema precisa fazer antes de qualquer linha de código ser escrita.

Essa etapa tem um nome próprio: **Engenharia de Requisitos**.

O processo de Engenharia de Requisitos

A Engenharia de Requisitos é dividida em quatro processos que se repetem ao longo do projeto:



- **Elicitação:** conversar com quem conhece o problema para descobrir necessidades.
- **Especificação:** transformar o que foi descoberto em documentação clara.
- **Validação e negociação:** confirmar com o cliente se entendemos corretamente, ajustando conflitos.
- **Gerenciamento:** controlar como os requisitos mudam ao longo do projeto (e eles sempre mudam).

Por que gerar diferentes visões da especificação? Porque um sistema tem vários interessados (*stakeholders*), e cada um enxerga o problema de um jeito:

Visão	Quem participa	Foco
Requisitos de Usuário	gestores, usuários finais, clientes	O quê o sistema faz — linguagem simples, sem jargão técnico
Requisitos de Sistema	analistas, arquitetos, desenvolvedores	Detalhamento técnico — completo, consistente, parte do contrato

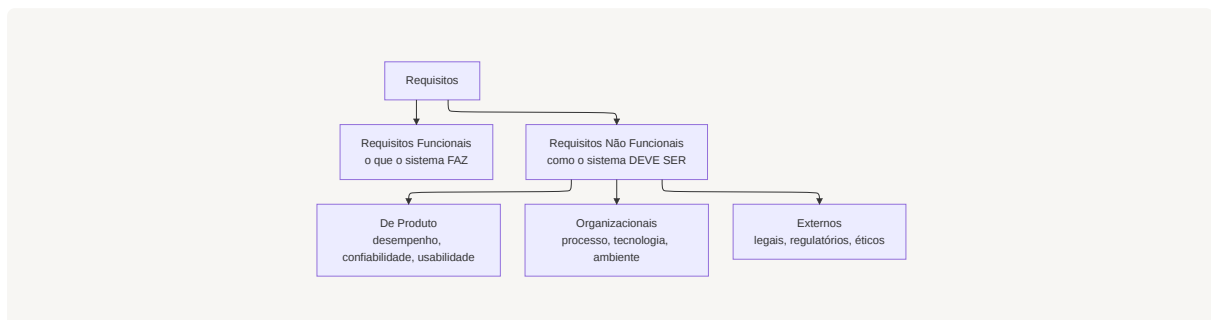
Exemplo — Sistema de Gestão Hospitalar:

- Requisito de usuário: *"O sistema deve permitir que a recepção veja quais médicos estão disponíveis para consulta em um determinado dia."*
- Requisito de sistema (mesmo requisito, detalhado): *"O sistema deve consultar a agenda de todos os médicos cadastrados na especialidade selecionada e retornar, em até 2 segundos, os horários livres para a data informada."*

Note que o requisito de sistema já entra em detalhes técnicos que o gestor do hospital nem precisa conhecer — mas que a equipe de desenvolvimento precisa ter muito claros.

Requisitos Funcionais x Requisitos Não Funcionais

Todo requisito se encaixa em uma dessas duas categorias:



Requisitos Funcionais (RF)

Descrevem **serviços** que o sistema oferece: o que ele deve fazer diante de determinadas entradas, e como deve se comportar em situações específicas. Em geral, começam com um verbo no infinitivo.

Exemplos — SGH:

- Cadastrar paciente com nome, CPF, data de nascimento e convênio
- Agendar consulta médica por especialidade e horário disponível
- Registrar prescrição de medicamentos vinculada a uma consulta

- Gerar relatório mensal de atendimentos por clínica
- Emitir alerta quando um paciente tiver alergia a um medicamento prescrito

Requisitos Não Funcionais (RNF)

Não descrevem uma função específica — descrevem **restrições** sobre como o sistema deve se comportar como um todo (desempenho, segurança, confiabilidade, usabilidade).

Exemplos — SGH:

- O sistema deve responder a uma busca de paciente em até 2 segundos
- O sistema deve estar disponível 24 horas por dia, incluindo finais de semana
- Apenas usuários autenticados com CPF e senha podem acessar o prontuário do paciente
- O sistema deve seguir as normas de proteção de dados de saúde (LGPD)

Atenção: um requisito não funcional malfeito pode inutilizar o sistema inteiro. Um usuário consegue contornar uma função que falha, mas não consegue contornar um sistema lento demais ou inseguro.

Os três tipos de Requisito Não Funcional

Tipo	O que restringe	Exemplo no SGH
De produto	comportamento do próprio software	"O sistema deve suportar 200 usuários simultâneos sem perda de desempenho"
Organizacional	políticas da empresa cliente ou desenvolvedora	"O sistema deve ser desenvolvido seguindo o padrão de código da equipe de TI do hospital"
Externo	fatores fora do sistema e do processo	"O sistema deve atender à Lei Geral de Proteção de Dados (LGPD) para dados de pacientes"

Regra de Negócio

As **regras de negócio** são restrições, políticas ou condições definidas pela organização que determinam como o sistema deve funcionar para atender às

necessidades do negócio. Elas representam decisões e normas que devem ser obrigatoriamente respeitadas durante a execução dos processos.

Exemplo:

- O sistema não deve permitir o cadastro de dois usuários com o mesmo CPF.
- Um pedido só poderá ser faturado após a confirmação do pagamento.
- Apenas usuários com perfil de administrador podem excluir registros.

Importante: As regras de negócio descrevem **o que deve ser obedecido pelo sistema**, independentemente da tecnologia utilizada para implementá-lo. Elas garantem que os processos sejam executados de acordo com as políticas e objetivos da organização.

Escrevendo requisitos de qualidade

Um bom requisito precisa ser **testável** — ou seja, deve ser possível verificar, de forma objetiva, se ele foi atendido.

Requisito ruim:

"O sistema deve ser fácil de usar pela equipe de enfermagem e minimizar erros de digitação."

Problema: o que é "fácil"? Como medir "minimizar"? Não há critério objetivo.

Requisito bom (reescrito):

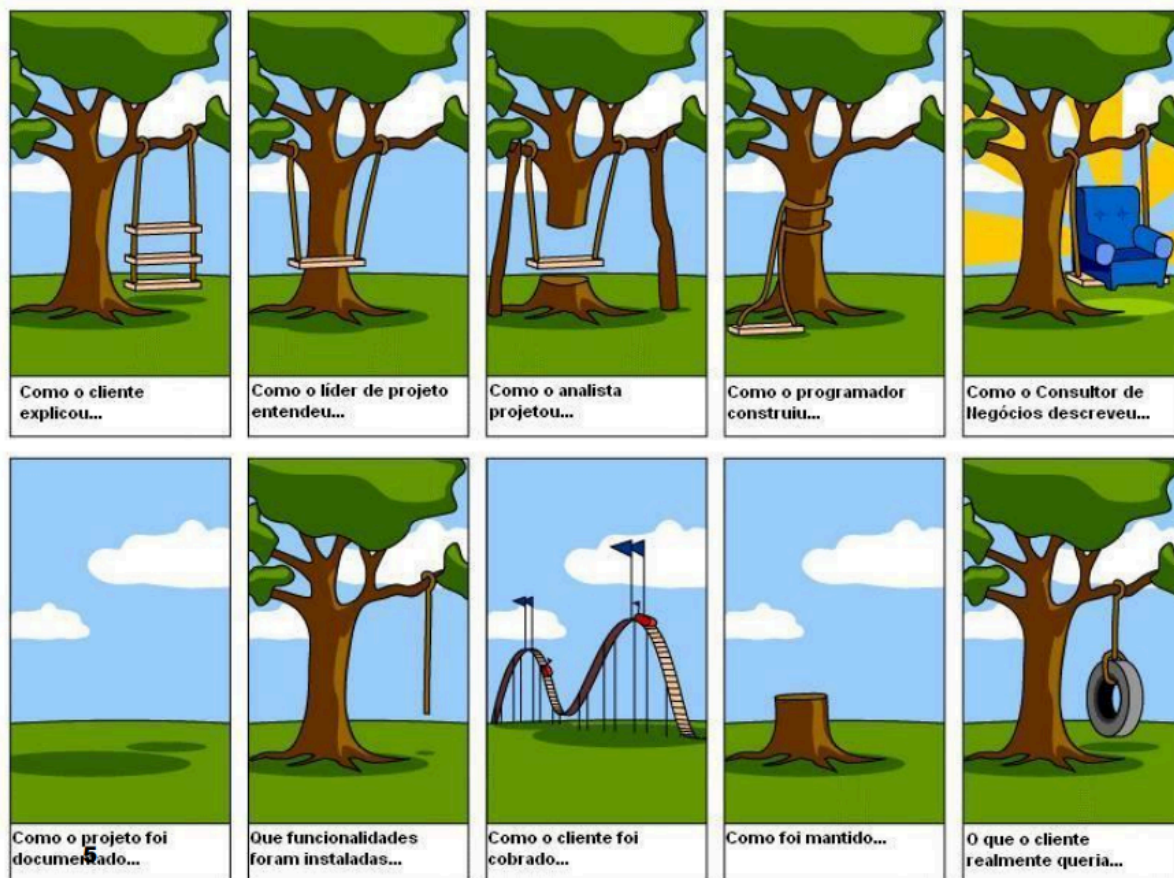
"A equipe de enfermagem deve conseguir registrar os sinais vitais de um paciente em até 2 minutos, após 4 horas de treinamento no sistema. O número médio de erros de digitação não deve ultrapassar 1 a cada 20 registros."

Agora dá para testar: cronometrar, contar erros, comparar com a meta.

Métricas úteis para transformar metas vagas em requisitos testáveis:

Característica	Métrica possível
Velocidade	transações processadas por segundo, tempo de resposta
Tamanho	megabytes ocupados, número de registros suportados
Facilidade de uso	tempo de treinamento, número de erros por hora de uso

Característica	Métrica possível
Confiabilidade	tempo médio entre falhas, percentual de disponibilidade
Robustez	tempo de reinício após falha, % de eventos que causam falha



Técnicas de elicitação de requisitos

Como descobrir os requisitos na prática? As duas técnicas mais comuns no início de um projeto:

Entrevistas: conversas estruturadas com stakeholders (no nosso caso: recepcionistas, médicos, gestor do hospital) para entender processos atuais, dificuldades e expectativas. Boas entrevistas misturam perguntas fechadas ("quantos pacientes vocês atendem por dia?") com perguntas abertas ("o que mais atrapalha o atendimento hoje?").

Cenários: descrições narrativas de como um usuário interage com o sistema em uma situação concreta, do início ao fim. Ajudam a revelar requisitos que ninguém lembraria de mencionar isoladamente.

Brainstorming: Técnica de levantamento de requisitos baseada na geração livre de ideias entre os participantes. O objetivo é reunir diferentes perspectivas sobre o sistema, analisar as sugestões e definir quais necessidades farão parte do escopo do projeto.

Exemplo de cenário — SGH:

Maria chega à recepção do hospital para uma consulta com o cardiologista. A recepcionista busca o cadastro de Maria pelo CPF. O sistema mostra que ela já é paciente cadastrada e exibe seu histórico de consultas anteriores. A recepcionista confirma a presença de Maria para o horário agendado. O sistema atualiza o status da consulta para "em atendimento" e notifica o consultório do médico.

Note como esse cenário, sozinho, já sugere vários requisitos funcionais: buscar paciente por CPF, exibir histórico, confirmar presença, atualizar status, notificar consultório.

Exemplo: Sistema de Gestão Hospitalar (SGH)

Briefing do cliente:

Um hospital de médio porte precisa informatizar sua gestão de pacientes. Hoje, o cadastro é feito em papel e a agenda de consultas é controlada em uma planilha, o que já causou perda de informações e conflitos de horário. O hospital quer um sistema que permita cadastrar pacientes, agendar consultas por especialidade, registrar o histórico de atendimentos (diagnósticos, prescrições e tratamentos), controlar internações e gerar relatórios gerenciais mensais para a diretoria. O sistema será usado por recepcionistas, médicos e pela equipe de enfermagem, e futuramente pode precisar trocar dados com um sistema de convênios médicos externo.

Do requisito ao caso de uso

Um conjunto de requisitos relacionados normalmente representa uma **tarefa completa** que um usuário realiza com o sistema — isso é um **caso de uso**. Vamos formalizar e desenhar esses casos de uso na próxima aula, mas já podemos começar a identificá-los agora.

Atores candidatos no SGH:

Ator	Tipo	Papel
Recepcionista	Principal	Cadastra pacientes, agenda e confirma consultas
Médico	Principal	Registra diagnóstico, prescreve medicação e tratamento
Enfermeiro(a)	Principal	Registra sinais vitais e acompanha internações
Paciente	Secundário	Interage indiretamente, via recepcionista
Sistema de Convênios (externo)	Secundário	Troca dados de cobertura com o SGH

Mapeando requisitos para casos de uso candidatos:

Requisitos relacionados	Caso de uso candidato
Cadastrar paciente com nome, CPF, data de nascimento e convênio	Cadastrar Paciente
Agendar consulta por especialidade e horário disponível; confirmar presença do paciente	Agendar Consulta
Registrar diagnóstico, prescrição e tratamento vinculados à consulta	Registrar Atendimento
Controlar entrada, acompanhamento e alta de pacientes internados	Gerenciar Internação
Gerar relatório mensal de atendimentos por clínica	Gerar Relatório Gerencial

Isso é exatamente o que vamos transformar em diagrama de casos de uso nas próximas aulas

Prática – Levantamento de Requisitos

Agora que estudamos o processo de Engenharia de Requisitos, chegou o momento de iniciar a documentação do projeto que será desenvolvido ao longo do semestre.

Com base no tema escolhido, realize o levantamento inicial dos requisitos do seu sistema. Nesta etapa, seu objetivo é identificar as principais funcionalidades, restrições e regras de negócio, documentando-as em **Linguagem Estruturada**, conforme apresentado em aula.

Organize sua documentação contendo:

- **Requisitos Funcionais (RF);**

- **Requisitos Não Funcionais (RNF);**
- **Regras de Negócio (RN).**

Utilize a seguinte estrutura para documentar cada requisito.

Exemplo de Requisito Funcional

RF-01

Nome: Gerar relatório de medicamentos

Descrição: No último dia útil de cada mês, o sistema deve gerar um relatório contendo os medicamentos prescritos, seus respectivos custos e as prescrições realizadas por cada clínica.

Exemplo de Requisito Não Funcional

RNF-01

Nome: Disponibilidade do sistema

Descrição: O sistema deve permanecer disponível durante o horário de funcionamento (segunda a sexta-feira, das 8h30 às 17h30). Períodos de indisponibilidade não podem ultrapassar 5 segundos por dia.

Exemplo de Regra de Negócio

RN-01

Nome: Cadastro único de usuários

Descrição: O sistema não deve permitir o cadastro de dois usuários com o mesmo CPF.

Orientações

- Escreva requisitos **claros, objetivos e testáveis**.
- Evite descrições vagas, como "o sistema deve ser rápido" ou "o sistema deve ser fácil de usar". Sempre que possível, utilize métricas.
- Numere todos os requisitos (RF-01, RF-02, RNF-01, RN-01...).
- Esta documentação será utilizada nas próximas aulas para construir os **Casos de Uso, Diagramas UML** e os demais artefatos do projeto. Portanto, ela poderá ser complementada e refinada ao longo do semestre.

Entrega: O documento será desenvolvido ao longo de todo o semestre e entregue apenas ao final da disciplina. Escolha o editor de texto de sua preferência e mãos à obra!

Aula 3 e 4 — Modelagem de Casos de Uso: Elementos e Primeiros Diagramas

Do requisito ao diagrama

Na aula passada, a partir do briefing do Sistema de Gestão Hospitalar (SGH), levantamos requisitos, identificamos atores candidatos e agrupamos requisitos relacionados em casos de uso candidatos. Hoje vamos aprender a notação formal da UML para representar tudo isso visualmente: o **Diagrama de Casos de Uso**.

Um diagrama de casos de uso é composto por seis elementos de modelagem:

#	Elemento	Em uma frase
1	Ator	Quem ou o que interage com o sistema
2	Caso de Uso	O que o sistema faz por esse ator
3	Associação	A ligação entre ator e caso de uso
4	Relacionamento <<include>>	Reuso obrigatório entre casos de uso
5	Relacionamento <<extend>>	Comportamento opcional entre casos de uso
6	Generalização	Herança entre atores (ou entre casos de uso)

Vamos ver cada um deles com exemplos pequenos

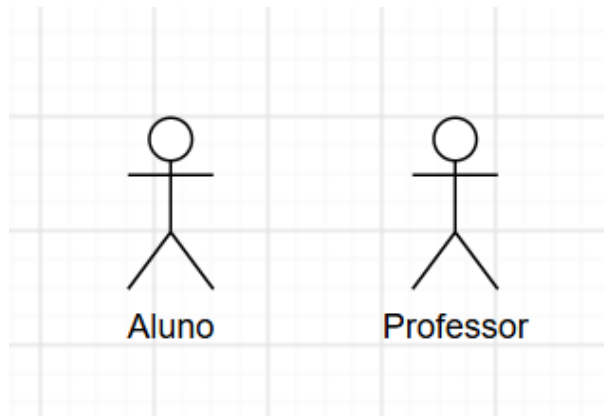
1. Ator

Um ator representa **um papel** desempenhado por alguém ou algo externo ao sistema que troca informações com ele. Não é necessariamente uma pessoa física — pode ser outro sistema, um dispositivo ou até uma organização.

Notação: um boneco-palito com o nome do papel embaixo. Quando o ator é um sistema externo (não uma pessoa), é comum representá-lo com um

retângulo estereotipado << sistema >> no lugar do boneco.

Regra importante: o nome do ator é sempre um **papel**, nunca uma pessoa específica. "João" não é um ator; "Recepcionista" é.



2. Caso de Uso

Um caso de uso representa **um comportamento-chave** do sistema — um objetivo completo que o ator alcança ao interagir com ele. O foco está na **finalidade**, nunca na implementação interna.

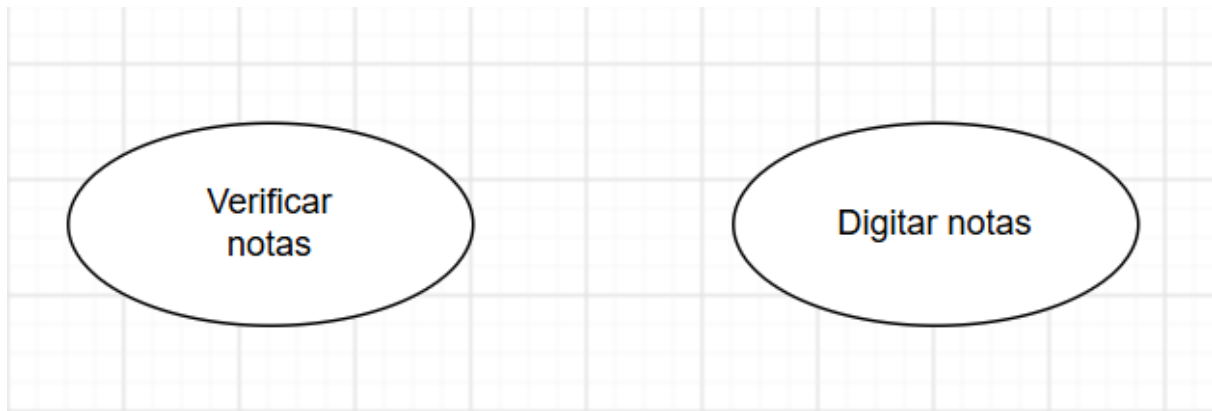
Notação: uma elipse, nomeada com um **verbo no infinitivo** + complemento.

Boas práticas de nomenclatura:

✗	✓ Nome bom	Por quê
"Tela de Login"	Autenticar Usuário	Nome de tela não é comportamento
"Botão Salvar"	Cadastrar Produto	Foca na ação da interface, não no objetivo
"Banco de Dados de Clientes"	Consultar Cliente	Isso é estrutura interna, não caso de uso

Casos de uso primários x secundários:

- **Primários:** representam os objetivos centrais do sistema — o motivo dele existir. Ex.: Realizar Venda , Matricular Aluno, Digitar notas, Verificar notas .
- **Secundários:** não trazem benefício direto ao ator, mas são necessários para o sistema funcionar. Ex.: Atualizar Cadastro de Produto , Fazer Backup dos Dados .

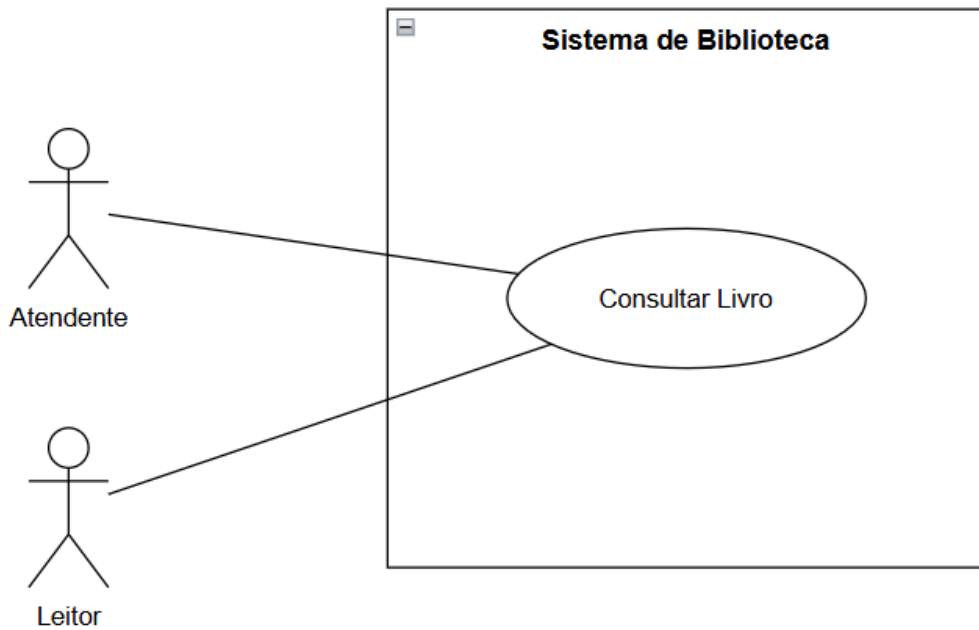


3. Associação

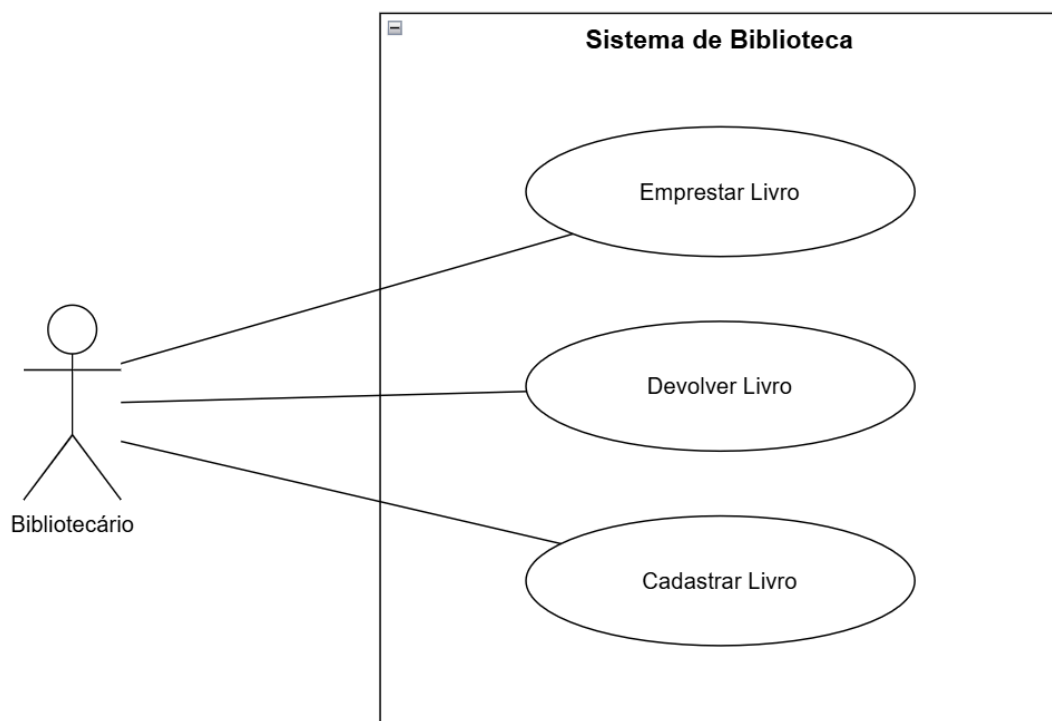
A associação é a linha simples (sem seta, na notação formal) que liga um ator a um caso de uso, indicando que eles trocam informações. Um mesmo caso de uso pode ser associado a mais de um ator, e um mesmo ator pode se associar a vários casos de uso.



Exemplo — um caso de uso, dois atores (para desenhar): em uma biblioteca, tanto o **Atendente** quanto o próprio **Leitor** (via totem de autoatendimento) podem realizar a **Consulta de Livro**.



Exemplo — um ator, vários casos de uso:



4. Relacionamento `<<include>>` (inclusão)

Existe **somente entre casos de uso** (nunca entre ator e caso de uso).

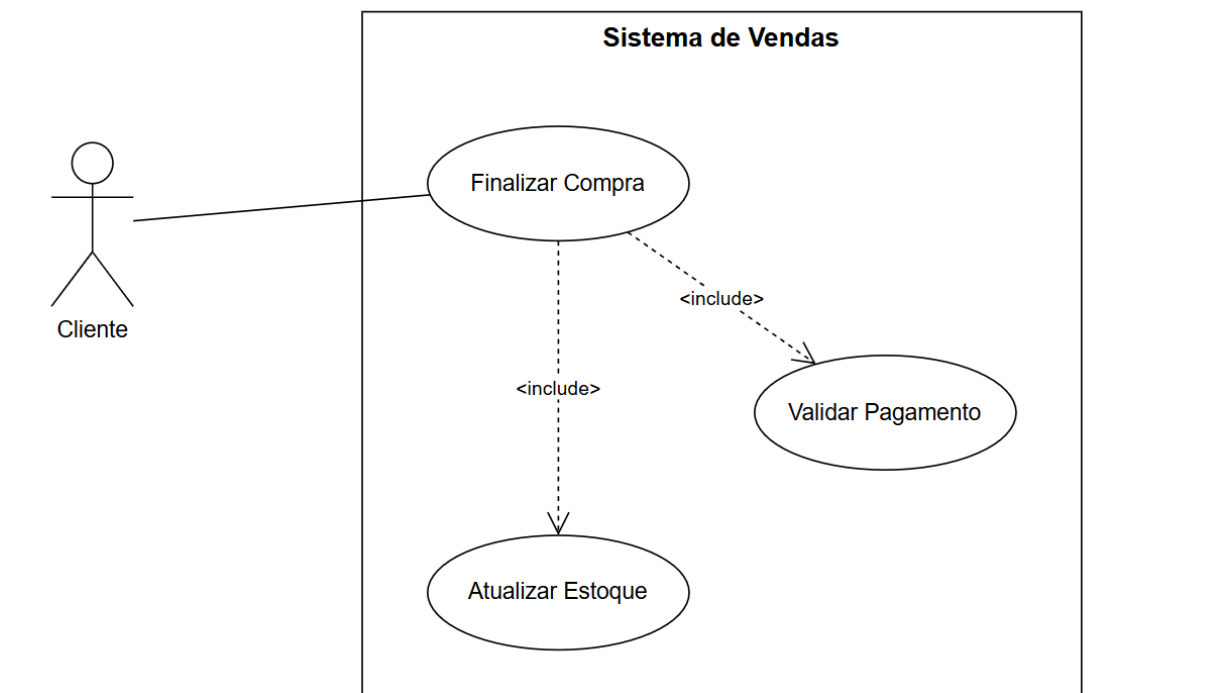
Representa uma parte do comportamento que se repete em mais de um caso

de uso — em vez de descrever a mesma sequência de passos várias vezes, extraímos essa parte comum para um caso de uso próprio, que passa a ser **reutilizado**.

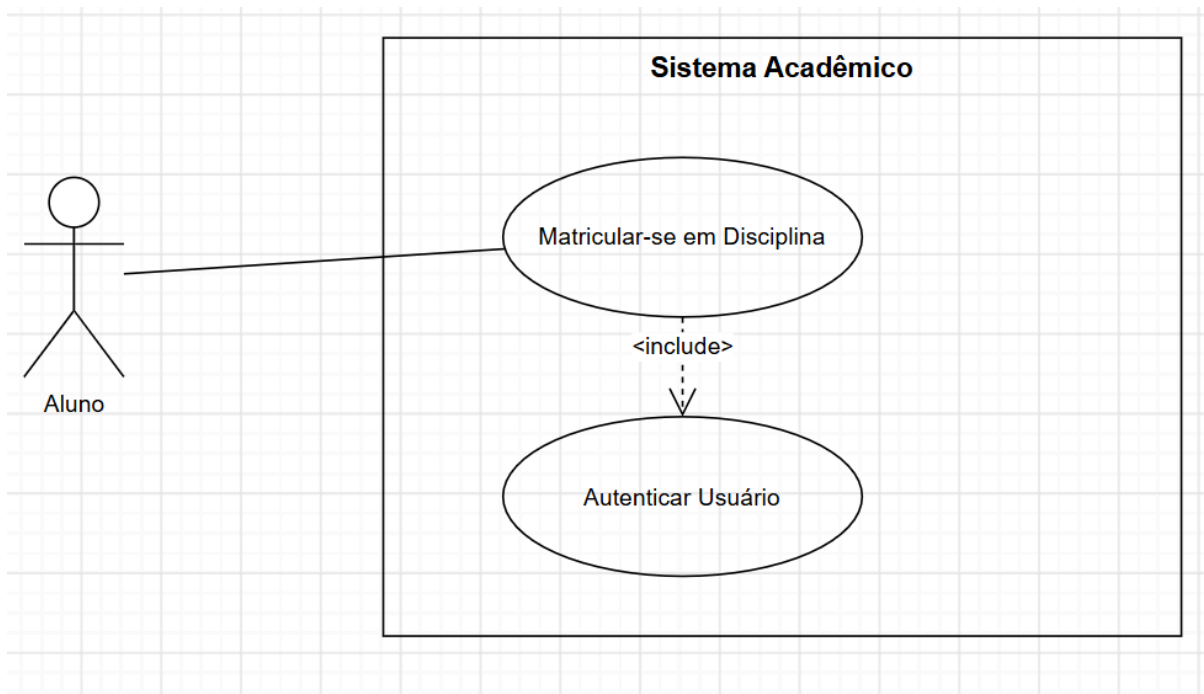
Regra de obrigatoriedade: se o caso de uso A inclui o caso de uso B, toda vez que A é executado, B também é executado — sem exceção.

Notação: seta tracejada com ponta aberta, apontando **do caso de uso base para o caso de uso incluído**, rotulada `<<include>>`.

Exemplo — sistema de vendas online:



Exemplo — sistema acadêmico: todo caso de uso que exige o usuário logado inclui `Autenticar Usuário`.



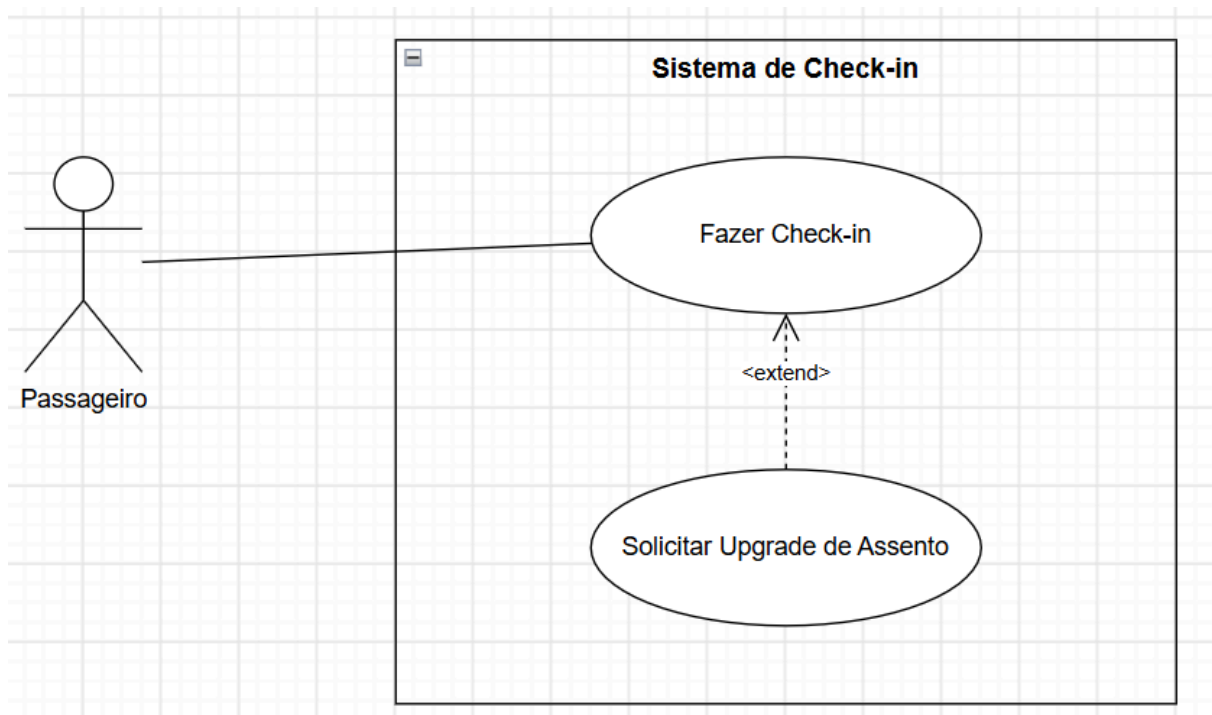
5. Relacionamento <<extend>> (extensão)

Também existe apenas entre casos de uso, mas representa o **oposto** da inclusão: um comportamento **opcional ou condicional**, que só acontece se uma determinada situação ocorrer. O caso de uso base funciona perfeitamente sem a extensão — ela apenas *"aumenta"* o comportamento quando aplicável.

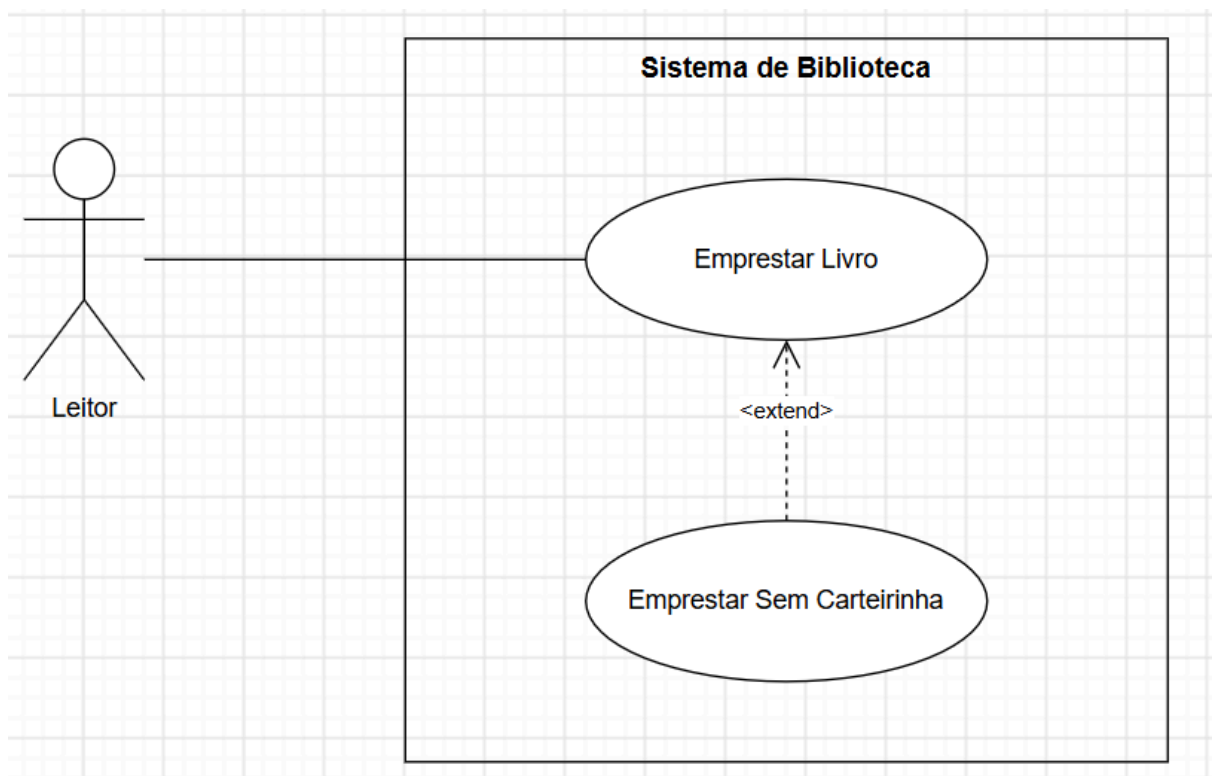
Atenção à direção da seta: ao contrário do <<include>>, a seta do <<extend>> é desenhada **do caso de uso de extensão para o caso de uso base**. É um erro comum inverter isso.

Notação: seta tracejada com ponta aberta, apontando da extensão para o caso base, rotulada <<extend>>.

Exemplo — check-in de aeroporto:



Exemplo — biblioteca: normalmente o empréstimo exige carteirinha, mas há um fluxo excepcional.



<<include>> x <<extend>> — resumo comparativo

	<<include>>	<<extend>>
Execução	Sempre obrigatória	Condicional, opcional
Direção da seta	Base → Incluído	Extensão → Base
Objetivo	Reaproveitar comportamento comum	Descrever variações e exceções
Pergunta-chave	"Isso sempre acontece junto?"	"Isso só acontece às vezes?"

6. Generalização

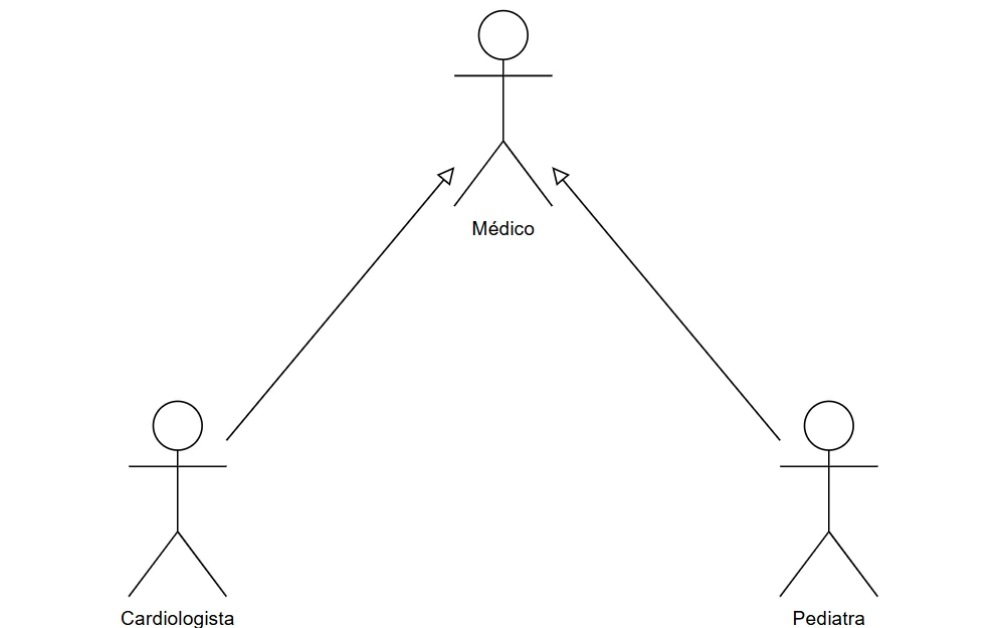
Representa uma relação de **herança** — mais comumente entre atores, mas também pode ocorrer entre casos de uso. Um ator especializado herda todas as associações do ator geral, além de poder ter as suas próprias.

Notação: seta sólida com ponta triangular vazada, apontando do ator específico para o ator geral.

A generalização é um relacionamento entre um ator A e um ator B. O ator A possui alguns casos de uso e ainda herda todas as características do ator B.



Exemplo — Hospital:



Isso significa: tudo que o **Médico** pode fazer no sistema, **Cardiologista** e **Pediatra** também podem — e cada um ainda pode ter casos de uso exclusivos (ex.: só o **Cardiologista** teria acesso a **Solicitar Exame Cardíaco**).

Fronteira do sistema

Um retângulo envolve todos os casos de uso e leva o nome do sistema no topo. Os atores ficam **do lado de fora**. Isso deixa explícito o que está dentro do escopo do sistema e o que é apenas um agente externo interagindo com ele — útil especialmente quando o cliente pergunta "isso está incluso no sistema ou não?".

Você já viu essa fronteira em todos os exemplos acima — é o retângulo ao redor das elipses.

Documentação do caso de uso

Um diagrama mostra **o quê**, mas não **como**. Por isso, todo caso de uso relevante deve vir acompanhado de uma descrição textual — geralmente em formato de tabela — detalhando o passo a passo da interação.

Modelo de documentação:

Campo	Conteúdo
Nome	Nome do caso de uso

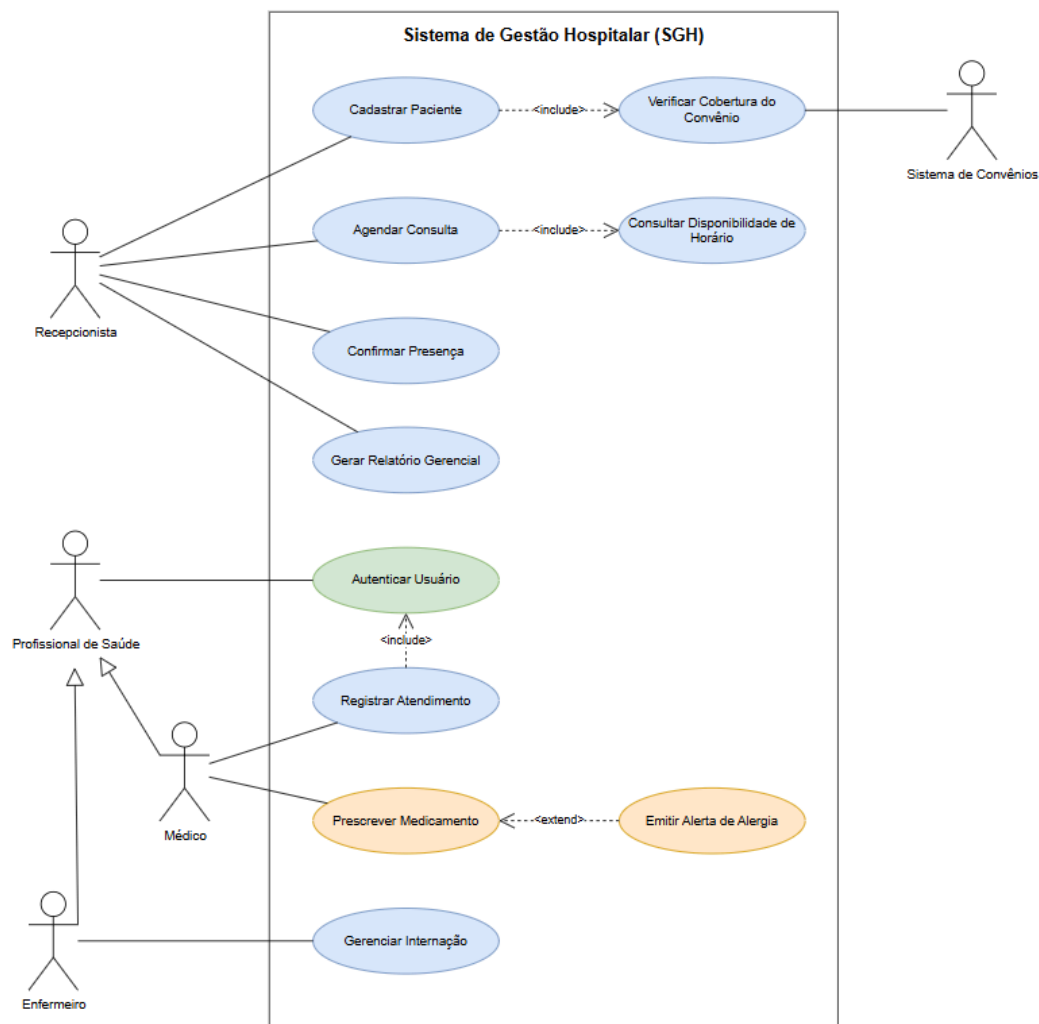
Campo	Conteúdo
Ator(es)	Quem participa
Descrição	Resumo em 1-2 frases
Pré-condições	O que precisa ser verdade antes de começar
Fluxo principal	Passo a passo do caminho "tudo dá certo"
Fluxos alternativos	Variações válidas do processo
Exceções	O que acontece quando algo dá errado
Pós-condições	O que passa a ser verdade depois de concluído

Exemplo completo — caso de uso **Agendar Consulta** (SGH):

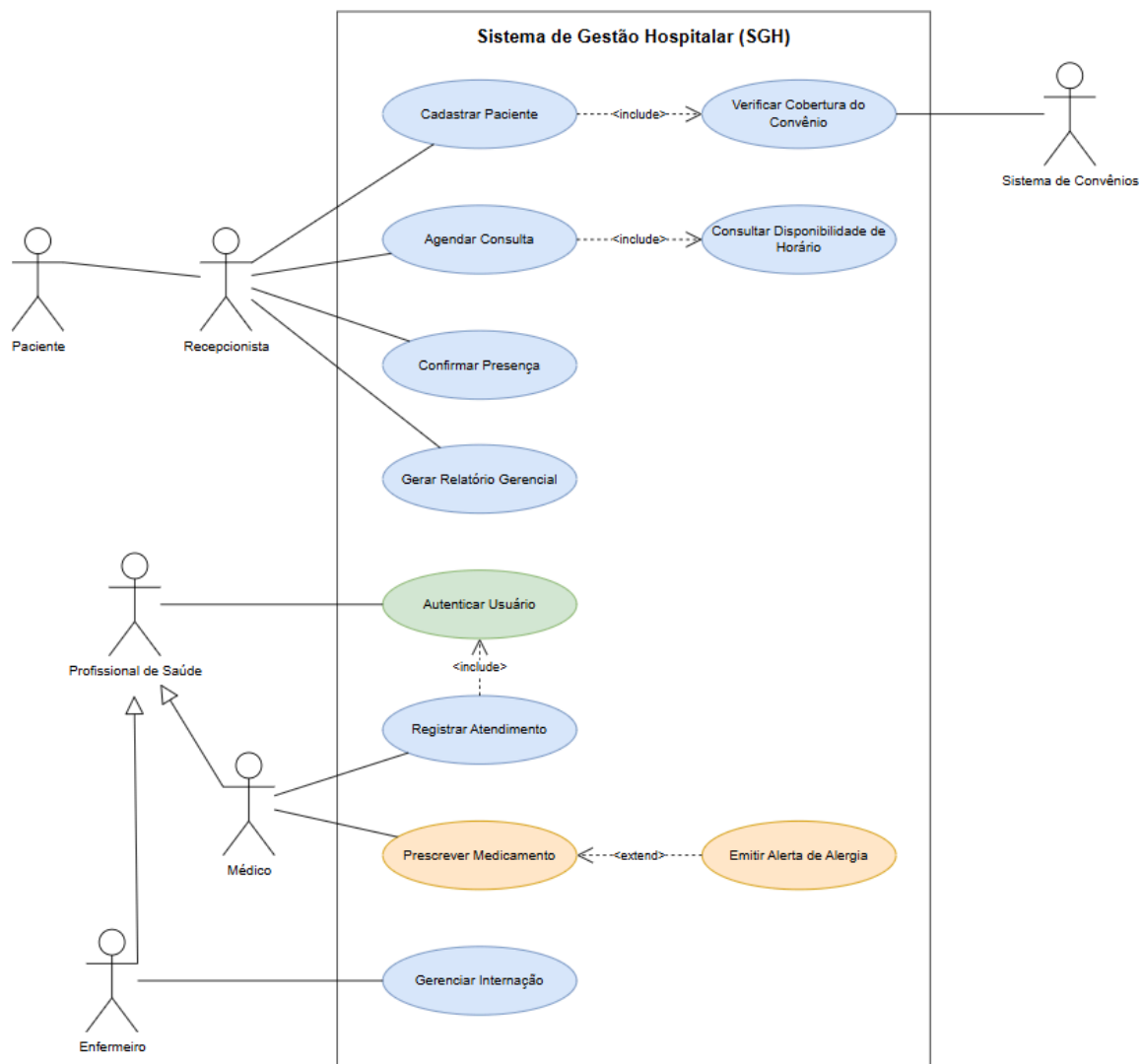
Campo	Conteúdo
Nome	Agendar Consulta
Ator(es)	Recepcionista (principal)
Descrição	A recepcionista agenda uma consulta médica para um paciente já cadastrado, em uma especialidade e horário disponíveis.
Pré-condições	O paciente já deve estar cadastrado no sistema
Fluxo principal	1. Recepcionista busca o paciente pelo CPF. 2. Sistema exibe os dados do paciente. 3. Recepcionista seleciona a especialidade desejada. 4. Sistema executa Consultar Disponibilidade de Horário e exibe os horários livres. 5. Recepcionista escolhe um horário. 6. Sistema confirma o agendamento e atualiza a agenda do médico.
Fluxos alternativos	3a. Se não houver horário disponível na especialidade escolhida, o sistema sugere a próxima data com vaga.
Exceções	1a. Se o CPF não for encontrado, o sistema oferece a opção de cadastrar um novo paciente antes de continuar.
Pós-condições	A consulta passa a constar na agenda do médico e no histórico do paciente

Diagrama de Casos de Uso do SGH

Agora vamos reunir tudo que vimos hoje e aplicar aos atores e casos de uso que identificamos na aula passada — incluindo a generalização entre **Médico** e **Enfermeiro**, que compartilham o papel geral de **Profissional de Saúde**.



Como ler: repare que **Prof** (Profissional de Saúde) é quem se autentica no sistema — e como **Médico** e **Enfermeiro** são generalizações dele, ambos herdam essa capacidade sem que precisemos redesenhar a associação duas vezes. Repare também que **Emitir Alerta de Alergia** só acontece condicionalmente (por isso <<extend>>), enquanto **Consultar Disponibilidade de Horário** acontece sempre que alguém agenda uma consulta (por isso <<include>>).



Instruções e exemplos para uso da ferramenta Draw.io

[Exemplo_Casos-De-Uso-UML.drawio](#)

Baixe o modelo e siga as instruções do professor para importar na ferramenta indicada.

Praticando conhecimentos:

Trecho de requisito ainda não modelado no SGH:

"Quando um paciente é internado, a enfermagem deve registrar os sinais vitais periodicamente. Se algum sinal vital sair da faixa normal, o sistema

deve gerar um alerta automático para o médico responsável. Ao final da internação, o médico registra a alta e o sistema calcula o tempo total de internação."

Em grupos, montem um diagrama de casos de uso para esse trecho, contendo:

1. Pelo menos **2 atores** (considere se algum já existe no diagrama do SGH)
2. Pelo menos **4 casos de uso**
3. **1 relacionamento** `<<include>>`
4. **1 relacionamento** `<<extend>>`
5. A **fronteira do sistema** desenhada

Depois, escolham **um** dos casos de uso do grupo e documentem-no usando a tabela (fluxo principal, alternativo e exceção).

Aula 5 — Classes de Análise: do Requisito ao Primeiro Modelo Estrutural

Contexto:

Diagrama de Casos de usos | Diagrama de Sequência → **Diagrama de Comportamento**

Diagrama de Classes → **Diagrama de Estrutura**

O que é uma classe?

Uma classe é o **molde estrutural** que define os atributos e métodos de um conjunto de objetos, sem representar nenhum objeto específico. Ela especifica **o que** cada objeto desse tipo terá (dados/estado) e **o que** poderá fazer (comportamento) a estrutura, não a instância.

Essa definição serve em dois momentos distintos do projeto:

- **No diagrama de classes:** a classe é usada para representar visualmente essa estrutura antes da implementação.
- **Na implementação:** a mesma classe passa a ser o modelo (template) a partir do qual cada objeto é instanciado.

Requisitos e casos de uso descrevem **comportamento**: o que o sistema faz. Agora entramos na visão **estrutural**: quais objetos existem no domínio do problema e como se relacionam. Essa visão é representada pelo **diagrama de classes**, que evolui em dois estágios:

	Classe de Análise	Classe de Projeto
Nível	Conceitual	Técnico
Origem	Requisitos e casos de uso	Classes de análise refinadas
Conteúdo	Nome, atributos, métodos em alto nível	Visibilidade, tipos de dados, persistência, padrões de projeto
Pergunta que responde	"Quais conceitos existem no domínio?"	"Como isso será implementado?"
Exemplo	Paciente (nome, cpf)	Paciente (-nome: String, -cpf: String, +validarCPF(): boolean)

Primeiramente vamos focar em **classe de análise**. O refinamento para classe de projeto vem em aulas futuras.

Identificando classes de análise

Técnica padrão: extrair **substantivos candidatos** dos requisitos e das descrições de casos de uso, depois filtrar.

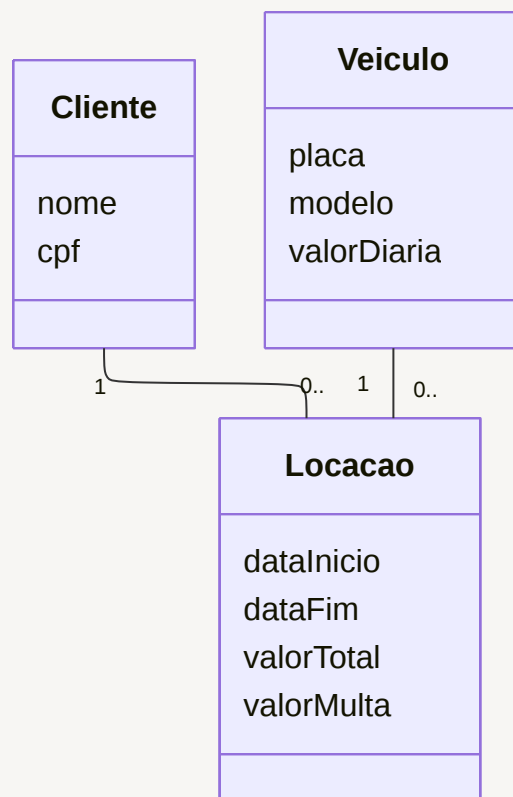
Heurísticas de filtragem:

- Tem atributos e comportamento próprios → possivelmente será uma classe
- É apenas um valor associado a outra coisa (data, quantidade, nome) → provavelmente é atributo
- Já é um ator do diagrama de casos de uso, mas o sistema precisa guardar dados sobre ele → também vira classe
- Termo fora do escopo do sistema → descarte
- Cuidado com sinônimos do mesmo conceito (Duplicidade)

Exemplo Locadora de Veículos:

"O sistema deve permitir que um Cliente alugue um Veículo por um Período determinado. Ao final da locação, o sistema calcula o Valor total com base na diária do veículo e nos dias utilizados. Se o veículo for devolvido com avarias, uma Multa adicional é registrada na locação."

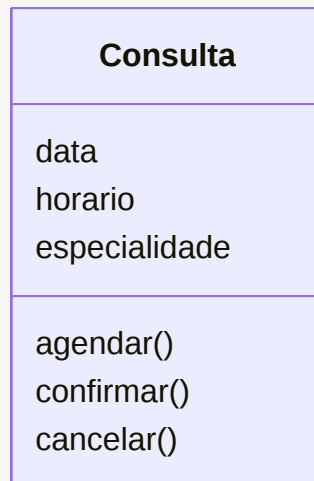
Substantivo candidato	Classe ou atributo?	Justificativa
Cliente	Classe	Tem dados próprios (nome, CPF) e participa de várias locações
Veículo	Classe	Tem dados próprios (placa, modelo) e existe independente de qualquer locação
Locação	Classe	Representa a transação entre Cliente e Veículo, com dados próprios (datas, valor)
Período	Atributo	Vira <code>dataInicio</code> / <code>dataFim</code> de Locação
Valor total	Atributo	Vira <code>valorTotal</code> de Locação
Multa	Atributo	Vira <code>valorMulta</code> de Locação não tem comportamento próprio que justifique uma classe separada



Estrutura de uma classe de análise

Três compartimentos: **nome**, **atributos**, **métodos**. Na análise, atributos aparecem sem tipo de dado e métodos sem assinatura completa, o foco é o

conceito, não a implementação.



Nem toda classe de análise precisa de métodos. Muitas representam apenas dados do domínio (ex.: `Cliente`, `Veiculo` acima) — os comportamentos de negócio costumam se concentrar nas classes que representam processos ou transações.

Associação entre classes

Uma linha simples liga duas classes relacionadas, com **multiplicidade** em cada ponta indicando quantos objetos participam da relação.

Notação	Significado
1	Exatamente um
0..1	Zero ou um
1..*	Um ou mais
0..* ou *	Zero ou mais

No exemplo da locadora: `Cliente "1" -- "0..*" Locacao` se lê como "um cliente pode ter zero ou mais locações; cada locação pertence a exatamente um cliente".

Esse é o relacionamento mais básico. Generalização (herança), agregação e composição têm regras próprias e serão vistas em detalhe na aula sobre Tipos de Relacionamentos.

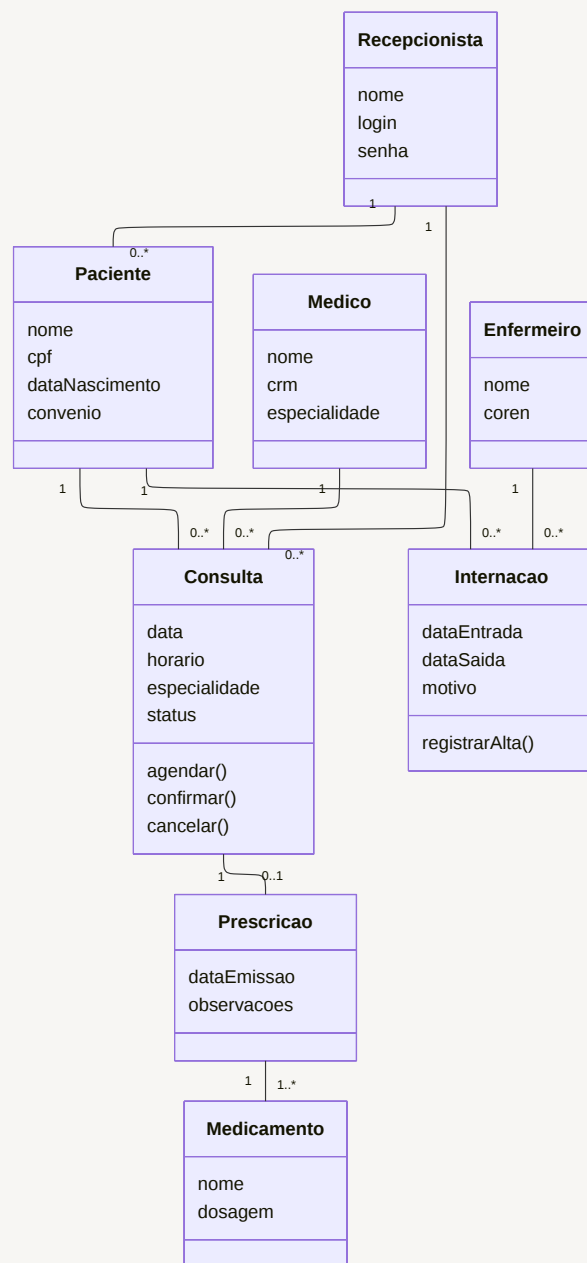
Aplicando ao SGH

Caso de uso / Requisito	Substantivos candidatos	Classe ou atributo?
Cadastrar Paciente (nome, CPF, data de nascimento, convênio)	Paciente	Classe — tem dados próprios e é referenciado por várias consultas
Agendar Consulta por especialidade e horário	Consulta	Classe — representa o evento de atendimento agendado
Registrar diagnóstico, prescrição e tratamento	Prescrição, Medicamento	Classes — uma prescrição agrupa um ou mais medicamentos
Gerenciar Internação	Internação	Classe — tem datas e motivo próprios
Médico e Enfermeiro autenticam no sistema	Médico, Enfermeiro	Classes — têm dados próprios (CRM, COREN, especialidade)
Recepcionista cadastra pacientes e agenda consultas — dados protegidos por LGPD	Recepcionista	Classe — pelo mesmo critério dos profissionais de saúde: o sistema precisa guardar dados sobre quem realiza essas ações

Ponto de atenção: um mesmo conceito do domínio pode aparecer como ator (visão de interação) e como classe (visão estrutural) — não é contradição, são perguntas diferentes sobre a mesma coisa. **Recepcionista** se qualifica como classe pelo mesmo motivo que **Médico** e **Enfermeiro**: o sistema precisa guardar dados sobre ela.

*(Repare que **Recepcionista**, **Medico** e **Enfermeiro** têm uma necessidade em comum — dados de acesso ao sistema. Isso é um indício de generalização, que vamos formalizar na aula de Tipos de Relacionamentos.)*

Diagrama de classes de análise do SGH:



Leitura: **Recepcionista** agora resolve a rastreabilidade de quem cadastra pacientes e agenda consultas — sem herança, sem estereótipo, só mais uma classe no mesmo nível das outras.

Prática: esboço do diagrama de classes do seu projeto

Usando os requisitos e casos de uso já documentados do seu projeto (individual ou em dupla):

1. Releia seus requisitos e a descrição dos seus casos de uso.
2. Grife os substantivos candidatos.
3. Aplique as heurísticas desta aula para separar classes de atributos.
4. Desenhe cada classe com nome, atributos e (quando fizer sentido) métodos em alto nível.
5. Conecte as classes com associações simples e multiplicidade, onde a relação for clara.

Este é um **esboço** não precisa de visibilidade, tipos de dados ou persistência ainda. Ele será a base para o diagrama de classes de projeto, que vamos evoluir nas próximas aulas.

Aula 6 e 7 — Diagrama de Classes: Notação Completa e Relacionamentos

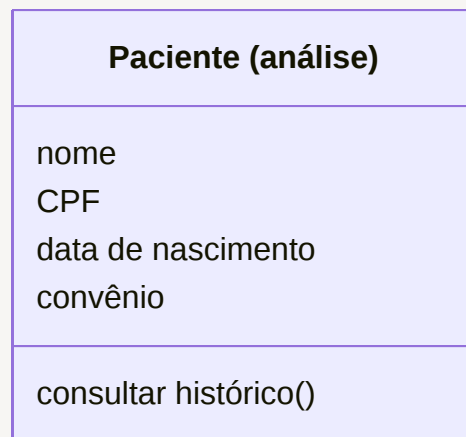
Onde o diagrama de classes se encaixa

Os diagramas UML se dividem em dois grupos: estruturais (o que o sistema **é**) e comportamentais (o que o sistema **faz**). Vocês já passaram pelo caso de uso (comportamental) e agora entram no diagrama de classes (estrutural) — que serve de base para os diagramas de sequência que virão depois.

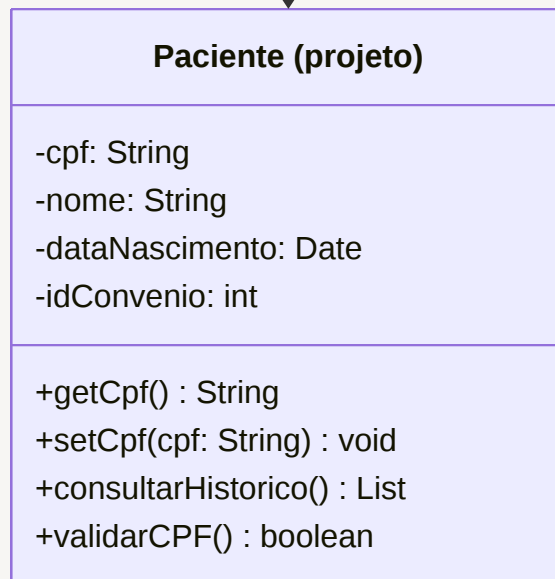
O diagrama de classes nasce dos casos de uso (o que o sistema precisa saber e fazer) e alimenta o diagrama de sequência (como os objetos trocam mensagens para executar cada caso de uso). Ele é a base de dados do projeto até o fim do semestre — não um exercício isolado.

Da classe de análise à classe de projeto

Vocês já esboçaram classes de análise: nome, atributos e métodos em alto nível, sem preocupação técnica. Hoje aprendemos a notação formal da UML — que é exatamente o que separa uma classe de análise de uma classe de projeto.



refinamento



A classe de análise é quase uma anotação de domínio. A classe de projeto usa a sintaxe formal que vamos ver agora.

Anatomia completa de uma classe

Toda classe é um retângulo de três compartimentos: nome, atributos, métodos.

Atributos: `visibilidade nome : tipo = valorInicial {propriedades}`

Símbolo	Visibilidade
+	public

Símbolo	Visibilidade
-	private
#	protected
~	package

Exemplos:

```
- cpf: String
```

```
+ PI: double = 3.1415 {final}
```

Métodos: `visibilidade nome(parâmetros) : tipoRetorno {propriedades}`

Exemplos:

```
+ validarCPF(): boolean
```

```
+ getNome(): String
```

Classes abstratas: nome em itálico ou anotação `<<abstract>>`. Nunca são instanciadas diretamente — só suas especializações. É o caso de `ProfissionalDeSaude`, que só existe através de `Medico` e `Enfermeiro`. Outro exemplo: `FormaDePagamento` só existe através de `Pix` e `CartaoCredito`, cada uma implementando `processar()` de um jeito diferente.

1. Exemplo classe Produto

Produto
- codigo: String - nome: String - preco: double = 0.0 + estoqueMinimo: int = 5 {readOnly}
+ aplicarDesconto(percentual: double): void + estaDisponivel(): boolean

Produto
+ codigo: String + nome: String + preco: double
+ aplicarDesconto(percentual: double): void + estaDisponivel(): boolean

1.1 Compartimento de atributos

Estrutura completa: `visibilidade nome : tipo = valorInicial {propriedades}`

Parte	O que representa	No exemplo
<code>visibilidade</code>	Quem pode acessar o dado de fora da classe (<code>+</code> público, <code>-</code> privado, <code>#</code> protegido, <code>~</code> pacote)	<code>-</code> em <code>preco</code>
<code>nome</code>	Identificador do dado, em camelCase	<code>preco</code>
<code>: tipo</code>	O tipo de dado armazenado	<code>double</code>
<code>= valorInicial</code>	Valor padrão atribuído na criação do objeto (opcional)	<code>= 0.0</code>
<code>{propriedades}</code>	Restrição adicional entre chaves (opcional)	<code>{readOnly}</code> em <code>estoqueMinimo</code> — não pode ser alterado depois de criado

1.2 Compartimento de métodos

Estrutura completa: `visibilidade nome(parâmetros) : tipoRetorno {propriedades}`

Parte	O que representa	No exemplo
<code>visibilidade</code>	Mesma lógica dos atributos	<code>+</code> — método público, chamado de fora da classe
<code>nome</code>	A ação que o objeto executa; verbo em camelCase	<code>aplicarDesconto</code>
<code>(parâmetros)</code>	Entradas necessárias para executar a ação, no formato <code>nome: tipo</code> , separadas por vírgula	<code>(percentual: double)</code>
<code>: tipoRetorno</code>	O que o método devolve; <code>void</code> quando não retorna nada	<code>void</code> / <code>boolean</code>
<code>{propriedades}</code>	Informação extra entre chaves (opcional)	<code>{abstract}</code> , <code>{static}</code>

Relacionamentos entre classes

Todo relacionamento tem três propriedades: **nome** (verbo que o descreve), **navegabilidade** (quem conhece quem) e **multiplicidade** (quantas instâncias participam).

Notação	Significado
1	Exatamente um
0..1	Zero ou um
* ou 0..*	Zero ou mais
1..*	Um ou mais
2..4	Faixa específica

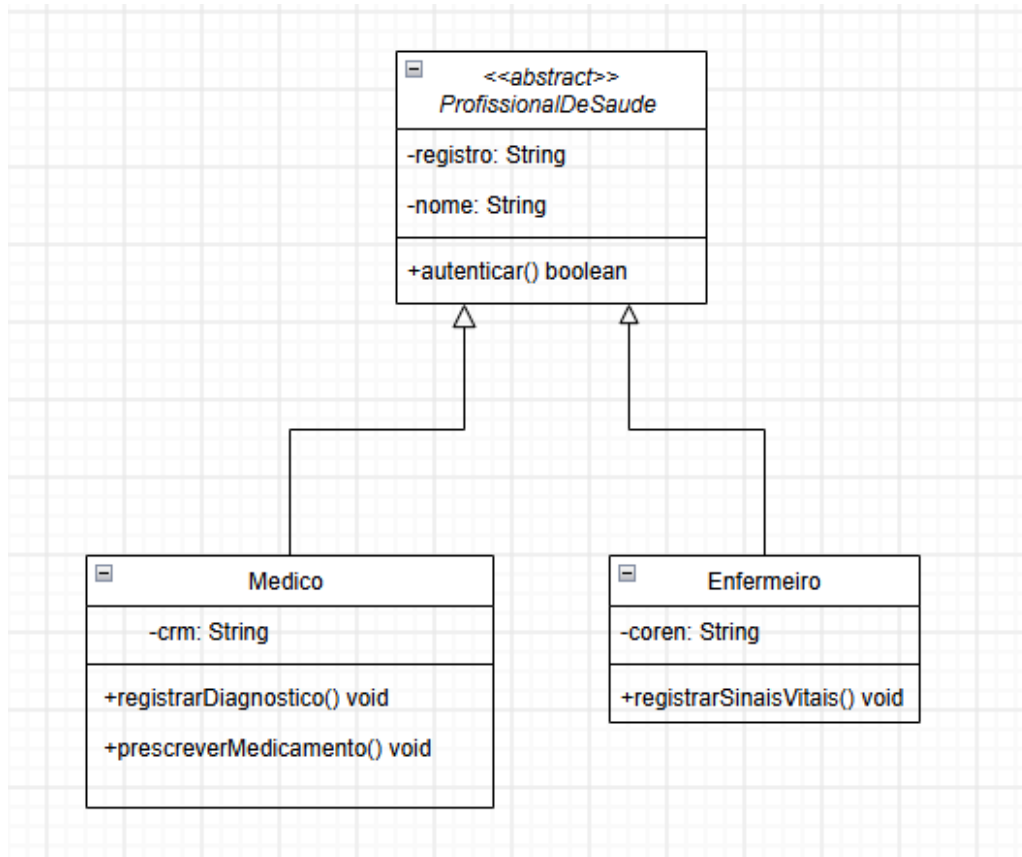
Generalização (herança) — "é um"

Relação "é um": a subclasse herda atributos e métodos da superclasse e pode especializar o comportamento.

Exemplo: `FormaDePagamento` (abstrata) → `Pix`, `CartaoCredito`. Ambas são formas de pagamento, mas cada uma processa a transação à sua maneira.

Notação: seta contínua com ponta triangular vazada, apontando da subclasse para a superclasse.

Mesmo princípio já visto entre atores no diagrama de casos de uso, agora entre classes:



A mesma regra de negócio, vista por dois ângulos diferentes: no caso de uso, como papéis; aqui, como estrutura de dados.

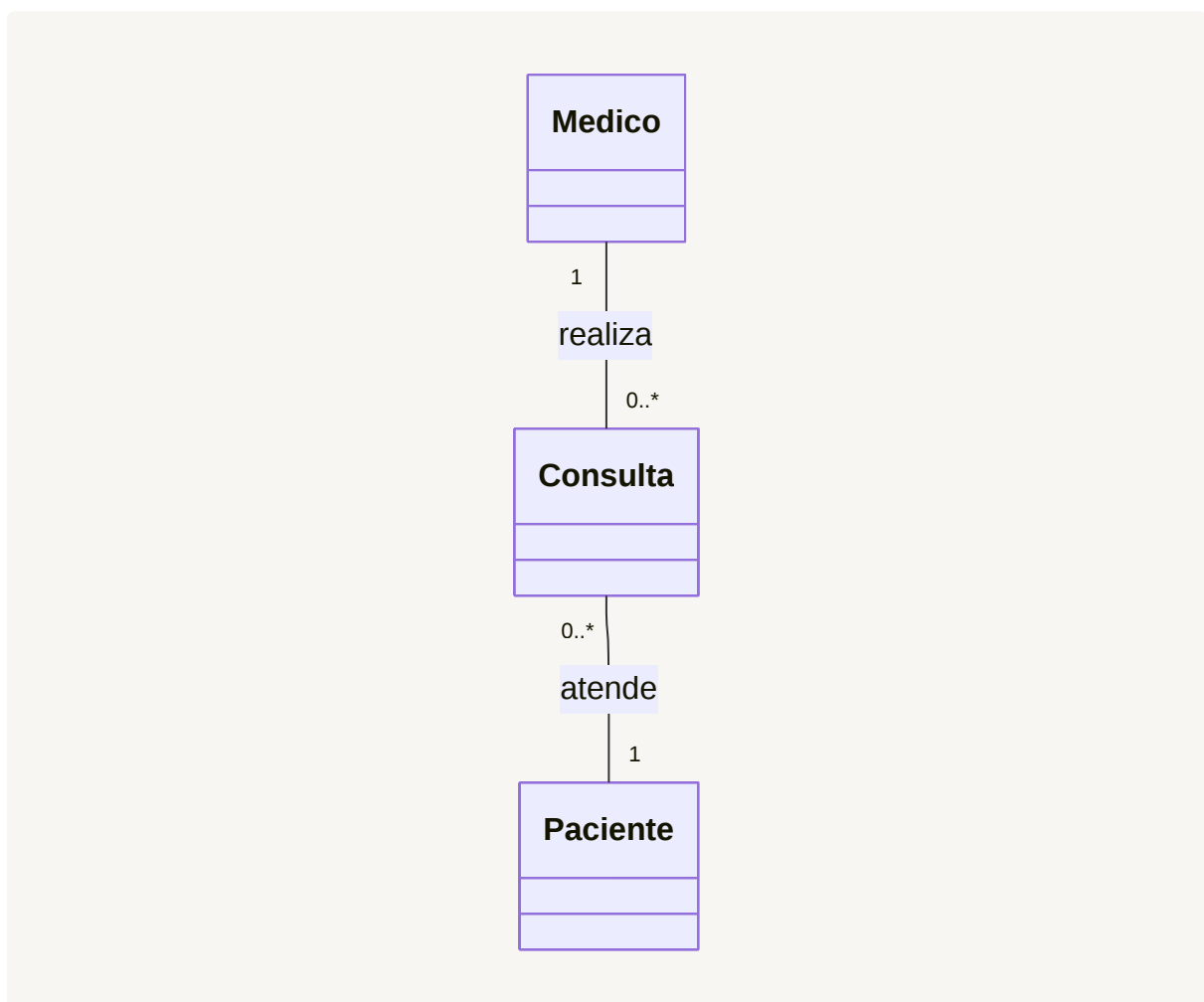
Associação — "usa"

A forma mais comum de relacionamento: uma linha sólida com multiplicidade em cada ponta.

Relação estrutural simples entre duas classes independentes: uma "conhece" ou "se relaciona" com a outra, sem que uma pertença fisicamente à outra. Ambas continuam existindo mesmo sem o vínculo.

Exemplo: `Aluno "1..*" -- "matricula-se em" -- "0..*" Curso`. Um aluno existe sem estar matriculado em nenhum curso; um curso existe sem ter alunos matriculados.

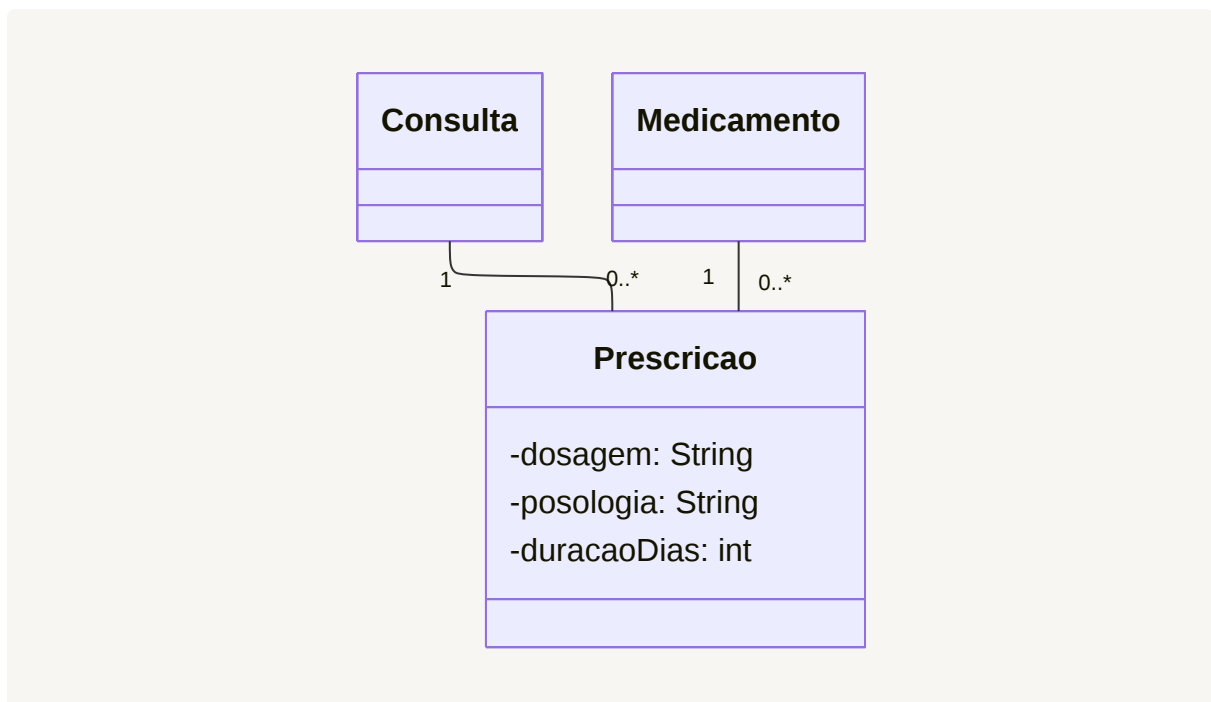
Notação: linha reta simples, com seta opcional indicando navegabilidade (quem consegue "chegar" no outro através do código).



Leitura: um médico realiza zero ou mais consultas; cada consulta atende exatamente um paciente.

Nuance técnica: quando o vínculo é temporário (ex.: um parâmetro de método), é tecnicamente uma **dependência** (seta tracejada >>>). Associação (linha sólida) é reservada a vínculos estruturais duradouros — um atributo que referencia outro objeto. Na prática do dia a dia isso costuma ser tratado de forma intercambiável, mas vale a diferença para quem quiser se aprofundar.

Classe associativa: quando o próprio relacionamento tem atributos, ele vira uma classe. É o caso da prescrição, que não faz sentido nem como atributo de **Consulta** sozinha, nem de **Medicamento** sozinho — ela existe na relação entre os dois.



Esse padrão é comum em sistemas de gestão (pedidos-produtos, matrículas-disciplinas etc.).

Agregação

Relação "todo-parte" fraca: o todo é composto por partes, mas as partes têm vida própria e podem existir e ser reaproveitadas fora do todo.

Exemplo: **Time** ◊— **Jogador**. Um jogador pode ser transferido para outro time e continua existindo normalmente.

Notação: losango vazado do lado da classe que representa o todo.

Composição

Relação "todo-parte" forte: as partes não têm existência própria fora do todo. Se o todo é destruído, as partes também são.

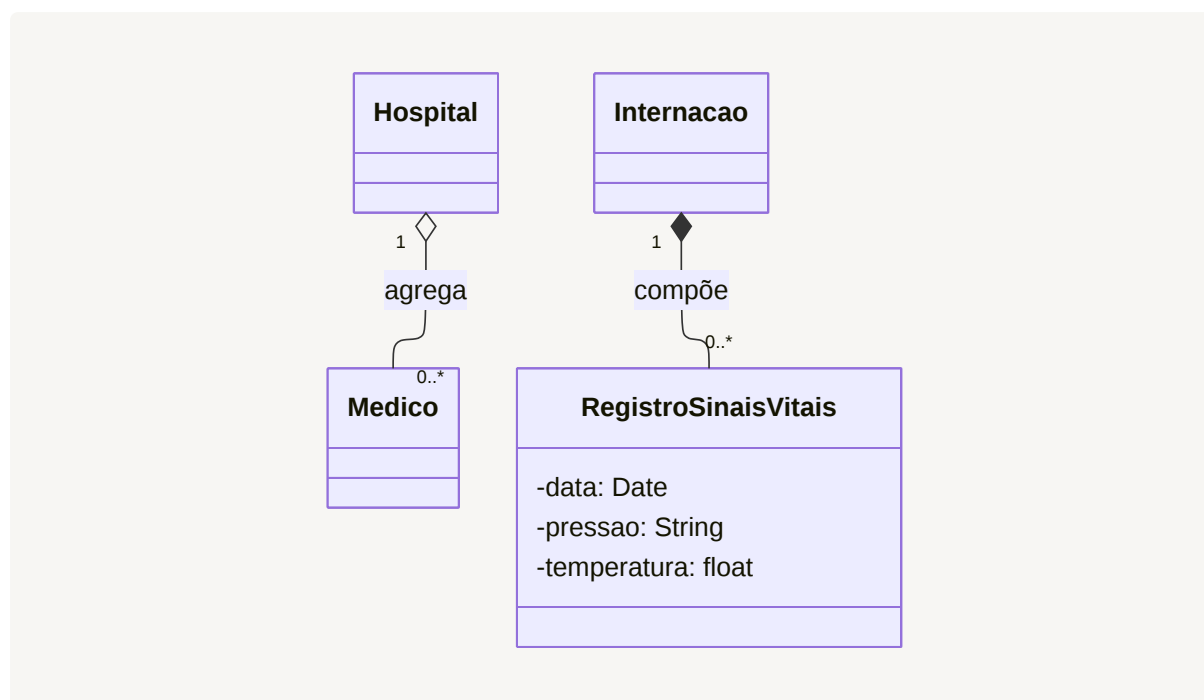
Exemplo: `Casa`  `Comodo`. Um cômodo não existe fora da casa a que pertence; se a casa é demolida, o cômodo deixa de existir.

Notação: losango preenchido do lado da classe que representa o todo.

Critério	Agregação	Composição
Losango	Vazado	Preenchido
Ciclo de vida da parte	Independente do todo	Depende do todo
Exemplo	<code>Time</code> e <code>Jogador</code>	<code>Casa</code> e <code>Comodo</code>
Pergunta-chave	"A parte sobrevive sem o todo?" → Sim	"A parte sobrevive sem o todo?" → Não

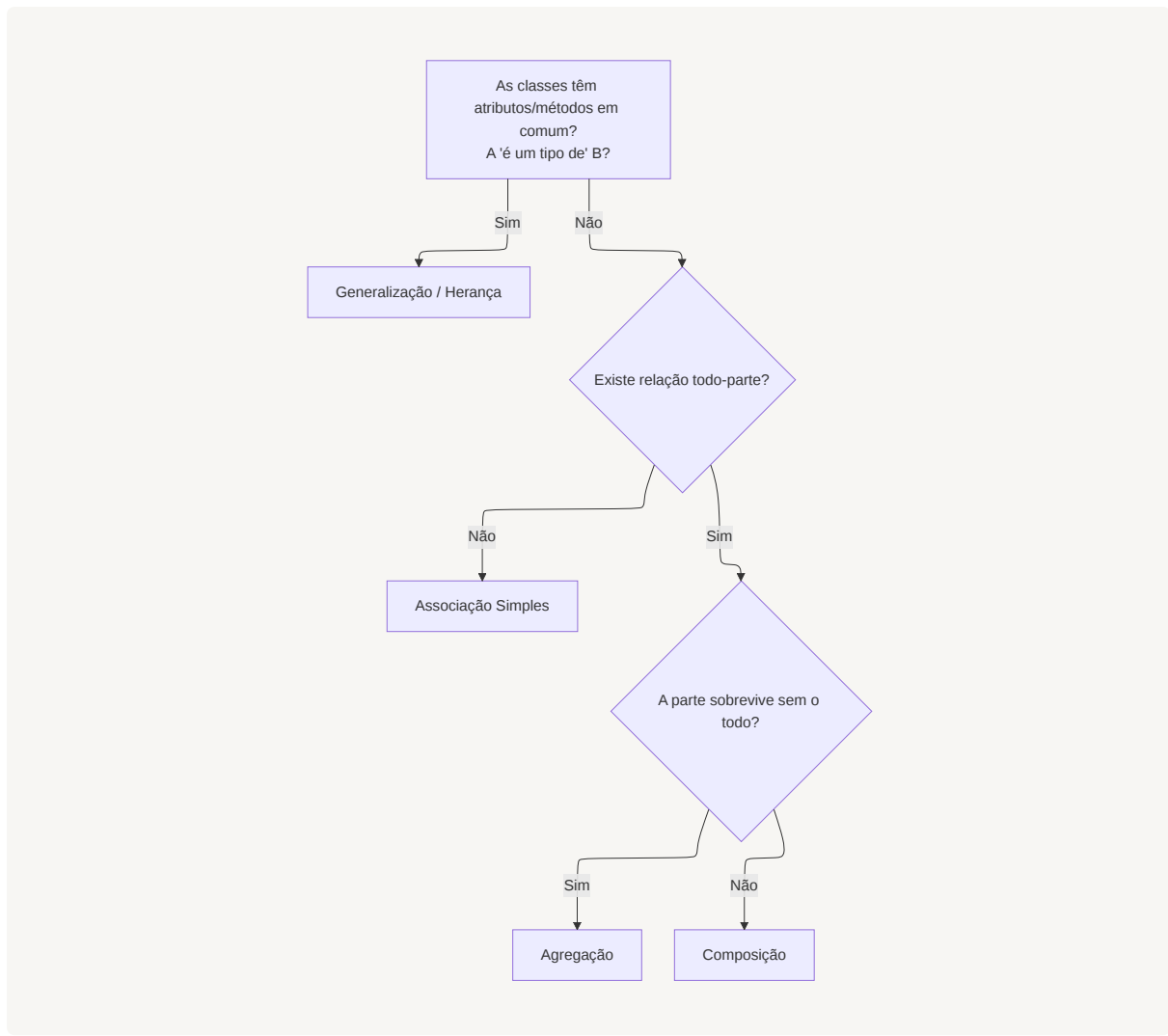
Agregação x Composição — "tem um" / "é parte de"

A pergunta que decide entre as duas: **a parte sobrevive sem o todo?**



- **Agregação (Hospital–Médico, losango vazado):** um médico existe independente do hospital — pode ser transferido, e o hospital fechar não faz o médico "deixar de existir" no sistema.
- **Composição (Internação–RegistroSinaisVitalis, losango preenchido):** se a internação for excluída, os registros de sinais vitais não fazem sentido isolados — são parte essencial dela.

Árvore de decisão



Atividade: refinando o esboço do seu projeto

Retome o esboço de classes que você desenhou na aula passada, a partir do seu próprio projeto:

1. Para cada linha de associação, classifique-a: associação simples, generalização, agregação ou composição — use a árvore de decisão.
2. Adicione multiplicidade em cada ponta que ainda não tiver.
3. Verifique se existe alguma **classe associativa** escondida como atributo em duas classes ao mesmo tempo.
4. Aplique o checklist de refinamento (dinâmico, estático, estrutural, nomenclatura, duplicidade).

Material complementar

Vídeo de referência indicado para revisão: <https://www.youtube.com/watch?v=rDidOn6KN9k>